Rapport Minishell

Yan Rabefaniraka

23 Mai 2025

Contents

Rappel de l'objectif . . . 1
Introduction . . . 1
Vue globale . . . 1
TP1 - Processus . . . 1
TP2 et TP3 - Signaux . . . 2
TP4 - Fichiers et redirections . . . 3
TP5 - Tubes . . . 4
Bilan . . . 4

Rappel de l'objectif

Ce projet était la mise en pratique des notions vues en TD / TP. Cela concernait notamment la gestion de processus, des signaux et des entrées et sorties. Pour ce faire, il a fallu, au long des différents TPs, implémenter les fonctionnalités de base d'un *minishell* (interpréteur de commandes).

Introduction

Ce bref rapport fera état de l'architecture de l'application et des choix de conception, et rendra compte des tests mis en place pour vérifier le bon fonctionnement des implémentations.

Vue globale

Notre programme est principalement constitué d'une boucle "infinie" où:

- on lit une ligne sur l'entrée standard du programme.
- En utilisant le programme fourni *readcmd*, on peut interpéter cette ligne comme une commande qui sera exécutée.
- On exécute la commande— soit en *background* (avec le suffixe *&*) soit en *foreground*— en créant un processus fils qu'on peut *tuer* (**Ctrl + C**) ou *suspendre* (**Ctrl + Z**) grâce à la gestion des signaux **SIGINT** et **SIGTSTP**.

Certaines commandes sont implémentées à la main car ne nécessitent pas de *fork()*; On parle de la commande *exit* pour fermer notre minishell, de *cd* pour changer de répertoire, et de *dir* qui liste les éléments du répertoire courant (~ à *ls*).

De plus, notre application supporte la redirection des entrées/sorties standards d'une commande à un fichier. Il aurait aussi du pouvoir effectuer cette redirection d'une commande à une autre à travers un procédé de *pipelines* qui n'a pas pu être implémenté pour faute de temps et de compréhension de l'impléméntation au sein du minishell.

TP1 - Processus

Dans ces étapes, on implémente la fonctionnalité basique d'exécution des commandes entrées en terminal.

On peut le tester avec des commandes basiques comme *ls* ou *cat*:

```
> ls
copier  Makefile  minishell  minishell.c  minishell.o  readcmd.c
readcmd.h  readcmd.o  test_readcmd.c
Le processus fils 1233344 a fini avec le code 0.
Processus père débloqué via le signal 17.

>
```

Ou encore:

```
> cat text.txt
Ceci est le texte contenu dans le fichier text.txt

Le processus fils 1233786 a fini avec le code 0.
Processus père débloqué via le signal 17.

>
```

→ On constate que les résultats sont bien les mêmes que dans le bash classique.

Ces étapes incluent la création du processus fils et la possibilité de le mettre en background.

On peut tester facilement son fonctionnement avec la commande *sleep* (c.f test dans la section suivante); → On voit que le processus de la 1re commande est lancé en *background* et nous permet donc de lancer la deuxième. On remarque aussi que ce 1*er* processus se termine bel et bien avant celui de la deuxième commande (en *foreground*) mais ne fait pas repasser le *minishell* en mode lecture active, comme attendu.

TP2 et TP3 - Signaux

Tout d'abord, en utilisant la primitive *waitpid* au sein d'un traitement de signal (un *handler*), on gère le déblocage du processus père via un signal. En accordance avec ce qui a été fait en TD, ce traitement permet aussi de débugger la fin du processus fils selon le signal reçu:

- ***WIFEXITED*** pour la fin du processus fils
- ***WIFSIGNALED*** pour l'interruption du processus fils (**CTRL + C** notamment)
- ***WIFSTOPPED*** pour la suspension du processus fils (**CTRL + Z** notamment)
- ***WIFCONTINUED*** pour la reprise d'un processus suspendu.

Cela nous permet entre autres d'attendre la fin terminaison du fils en avant-plan avec la primitive *pause()*. On peut de nouveau le tester avec *sleep*:

```
> sleep 10&
Commande 1235306 lancée en background.
> sleep 20
Le processus fils 1235306 a fini avec le code 0.
Processus père débloqué via le signal 17.

> Le processus fils 1235316 a fini avec le code 0.
Processus père débloqué via le signal 17.
```

On observe que le programme attend la fin du processus fils en avant-plan avant de reprendre le prompt (avec un petit problème d'ordre d'affichage je conçois).

Ensuite, on a implémenté la gestion des interruptions et suspensions manuelles avec les raccourcis **CTRL+C** et **CTRL + Z** (en *avant-plan uniquement*). L'implémentation finale gardée à été celle de la remarque du TP correspondant, avec la primitive *signal*, qui est beaucoup moins laborieuse à implémenter que les trois premières. Il est à noter que pour chacune des trois implémentations proposées (en commentaire dans le code), il semble y avoir

des comportements inattendus des raccourcis *i.e* —qui viennent probablement d'erreurs de programmation de ma part—nommément:

- Le minishell se met soudainement à accepter **CTRL + C** après certaines exécutions de commandes.
- Les raccourcis sont activés lors du lancement d'un processus avant-plan et permettent d'interrompre ce processus **et** le minishell ce qui était pourtant désactivé par la gestion des signaux / masque.

(Au moment de l'écriture de ce rapport je ne sais honnêtement plus quel problème a été ou non résolu pour ces trois procédés. Les tests qui suivent sont dans tous les cas pour la méthode implémentée.)

Le minishell ayant une command *exit* intégrée pour l'arrêter, il doit donc ignorer ces injonctions, là où ses processus fils doivent accepter leur comportement par défaut et (*resp.*) s'arrêter / se suspendre.

```
> *CTRL+C* ^C
> *CTRL+Z ^Z
```

Là où pour un processus fils on a:

```
> sleep 15 &
Commande 1238301 lancée en background.
> sleep 10 *CTRL+C*
^C
Le processus fils 1238311 a été tué avec le code 1.
Processus père débloqué via le signal 17.

>
Le processus fils 1238301 a fini avec le code 0.
Processus père débloqué via le signal 17.

>
```

Le groupement des processus et le traitement des signaux permet bel et bien de n'interrompre que le processus en avant-plan.

TP4 - Fichiers et redirections

Il nous est nécessaire de prendre en compte la redirection des entrées/sorties d'un processus afin de permettre à l'utilisateur d'y associer des fichiers. Pour ce faire, on utilise la primitive *open* avec les arguments *RD_ONLY* pour la lecture (entrée) et *O_WRONLY* | *O_CREAT* pour l'écriture (sortie) puis on duplique les descripteurs de fichiers. Les tests n'ont pas pu être réalisés pour cette fonctionnalité.

De plus, le comportement par défaut de la commande *cd* est quelque peu étrange sur le *minishell*. Il fallait donc l'implémenter manuellement (~l'override) à l'aide de la primitive *chdir*. On peut le tester ainsi:

```
> ls #on est dans le dossier SEC/minishell
copier    minishell    minishell.o  readcmd.h  readcmd.txt
Makefile  minishell.c  readcmd.c    readcmd.o  test_readcmd.c

Le processus fils 1240190 a fini avec le code 0.
Processus père débloqué via le signal 17.

> cd ..
> ls #on est bien dans le dossier SEC
copier.c  minishell  minishell-yra7512.tar  rapport_sec.md  TP1  TP2  TP3  tubes

Le processus fils 1240292 a fini avec le code 0.
Processus père débloqué via le signal 17.

>
```

Enfin, on implémente l'équivalent de la commande *ls* qui est la commande *dir*. On utilise les primitives *opendir*, *readdir*, *closedir* pour récupérer les éléments du répertoire donné en argument. Voici le test:

```
> dir #au sein du dossier SEC/minishell
.
..
readcmd.txt
readcmd.o
copier
minishell.o
Makefile
readcmd.h
test_readcmd.c
minishell.c
minishell
readcmd.c
> cd ..
> dir tubes
.
..
tube2
tube3
tubes_tmp
tube4.c
TP-Tubes.pdf
tube5.c
tube1
tube1.c
tube2.c
tube3.c
>
```

TP5 - Tubes

Comme dit dans l'introduction, cette partie du *minishell* n'a pas été implémentée malgré la complétion des 5 exercices intermédiaires.

Bilan

Pour finir, ce *minishell* permet le lancement de différents processus à travers les fils, et possède quelques commandes essentielles intégrées. Il gère aussi les envois de signaux pour permettre, non seulement de prendre en compte les interruptions et suspensions causées par l'utilisateur, mais aussi de veiller au bon déroulement des différents procédés d'avant-plan et d'arrière-plan ainsi que de la parenté des processus. Finalement il prend grossièrement en compte les fichiers en entrées et sorties standards des commandes. Pour autant, la mise en place de tubes et en particulier de *pipelines* entre les commandes n'a pas encore été mis en place. Dès lors, , on peut dire que ce *minishell* fait presque ce qu'on attend de lui.